



2025年度中国青年

揭榜挂帅擂台赛 · **华为赛道**



目录



1

赛题解读

2

评分规则

3

参考优化手段

4

答疑环节



赛题内容介绍



【赛题名称】推理大模型的训练调优与性能加速助力全栈自主AI

【赛题介绍】

在人工智能的时代浪潮中，各类大模型的发展突飞猛进，加速各行业应用创新。进入2025年，大语言模型（LLMs）快速迭代与进化，特别是在推理能力和运行效率方面有了显著提升。推理能力上，以OpenAI o1系列和DeepSeek R1为代表的推理模型，带来了scaling law范式的转移（inference-time scaling），非常擅长解决如解谜、高级数学和编码等复杂任务，将大模型推向更广泛的应用场景；运行效率上，以量化、MOE、知识蒸馏为代表的模型优化技术，使得大模型可以部署到更轻量化的设备上，给端侧AI带来更大的想象空间。

展开来看，DeepSeek证明了在没有思维链（Chain-of-Thought, CoT）监督数据的情况下，只通过强化学习也可以显著增强大语言模型的推理能力；而且，通过知识蒸馏，使用推理大模型产生的思维链数据和少量通用数据进行微调，小模型在资源受限的场景中也能实现复杂推理任务；此外，在参数量、算力需求指数倍增长情况下，如何充分释放AI芯片算力、提升大模型性价比也成为AI领域的关键技术，这其中高性能算子的实现与优化是达成此目标的基础。

因此，本次比赛聚焦大模型的推理能力提升和性能优化，要求选手基于华为全栈AI技术，使用强化学习、知识蒸馏等技术，提升选定的轻量级模型在数学计算、逻辑推理、代码生成等任务上的能力，并通过算子融合与优化、模型结构优化、量化等技术，保证精度的同时提升模型推理性能。

1. 本次比赛基于昇腾AI云服务，要求参赛团队使用AI芯片Ascend-snt9b、AI异构计算框架CANN、AI应用开发平台ModelArts等华为全栈AI技术，增强模型推理能力并提升性能。
2. 参赛团队需要掌握华为昇腾AI处理器架构、CANN软件栈与ModelArts开发环境，实现模型训练与推理调优，大赛将提供昇腾云模型开发、昇腾算子开发相关的技术文档及课程材料，助力参赛团队学习相关技术，了解实践操作。
3. 大赛鼓励选手将优化后的模型部署到开发板、手机等端侧设备上，以真实的产业需求为导向，探索推理模型在行业中的应用。

【赛题要求】

整体赛程分为初赛和总决赛，作品具体要求如下：

1. 初赛：组委会将提供Qwen轻量级模型与模型推理测试的示例工程，参赛团队自行选择强化学习算法（如GRPO、PPO等），或者知识蒸馏技术，在ModelArts的notebook开发环境（规格为Ascend: 1*ascend-snt9b(32G)|ARM: 24核 192GB）中对模型全部参数或部分参数进行微调。

比赛也考察参赛团队对大语言模型推理性能的调优能力，选手可以自行选择优化手段，如模型结构优化、算子融合、算子性能优化、量化等，在保证模型精度同时，尽可能提升推理性能。算子优化可以往小算子融合的方向设计（例如add+layernorm算子融合、matmul+add融合等）。

2. 总决赛：选手需准备方案介绍PPT进行答辩，内容包含推理能力提升和性能优化的技术方案、推理模型的应用demo等（可附上演示视频）。

酌情加分项：使用华为开发者空间提供的相关资源和服务、端侧应用鸿蒙适配、及其他华为开放能力调用。

目录



1

赛题解读

2

评分规则

3

参考优化手段

4

答疑环节



评分规则介绍

□ **格式:** 我们的测试用例是以jsonl格式组织的, 每行一条用例, 共有四种类型:

```
{"id": "id1", "type": "choice", "prompt": "某市老旧小区...需要多少天?", "choices": {"A": "9天", "B": "10天", "C": "11天", "D": "12天"}}
```

```
{"id": "id2", "type": "generic-generate", "prompt": "国际会议从北京时间9:15开始, 持续2小时50分钟。会议结束时是几点?"}
```

```
{"id": "id3", "type": "math", "prompt": "Express  $(5-4i)-2(3+6i)$  as a complex number."}
```

```
{"id": "id4", "type": "code-generate", "prompt": "from typing import List...    \"\"\"\\n\"\"\"}
```

大家返回的内容是:

```
{"result": {  
    "results": [  
        {"id": "id1", "content": "xxx"},  
        {"id": "id2", "content": ["<think>...</think><answer>...</answer>", "    return 1", "\treturn 1"]]},  
    }  
}
```

```
format_pattern = re.compile(r'^<think>(.*?)</think>\s*<answer>(.*?)</answer>$', re.DOTALL)
```

```
format_pattern.findall(content)
```

不支持后处理包装结果。最终得分归一化。



评分规则介绍

□ **精度**: 对于不同题目类型有不同的计算精度得分方式。

- ① choice: 提取答案内容: `r"(?i)Answer[ \t]*:[ \t]*\$( [A-D]* )\$(?)"`, 正确率: `extracted_answer == answer`
- ② generic-generate: 提取答案内容: `r"(?i)Answer\s*:\s*([^\n]+)"`, 匹配率: 生成的序列与真实序列具有相同的token序列的百分比。
- ③ math: 提取答案内容: `r"(?i)Answer\s*:\s*\s*([^\n]+)"`, 正确率: Math-Verify: `verify(answer_parsed, gold_parsed)`
- ④ code-generate: 对于不足3条的结果, 我们会使用已有结果随机扩展到3条, 对于超过3条的结果, 会随机采样3条。
提取答案内容: `r"```python\n(.*?)```"`, pass@k: 每个问题生成 k 个代码样本, 如果任何样本通过单元测试, 则认为问题已解决。

最终得分归一化。

□ **性能**: 性能排名按照tokens/s, 决赛会参考profiling等单算子性能/prefill/decode性能。最终得分归一化。

□ **总分**: 总分 = 0.4 * 精度得分 + 0.4 * 性能得分 + 0.2 * 格式得分

□ **评分环境**: Euler2.10.7, Cnn 8.1.RC1版本, Python3.9, 计算规格是Ascend: 1*ascend-snt9b(32G)|ARM: 24核 192GB。评分系统加载参赛团队提交的模型工程, 对比赛用的200条测试数据 (此部分数据不公开) 进行批量预测, 最后根据预测结果自动评分。

目录



1

赛题解读

2

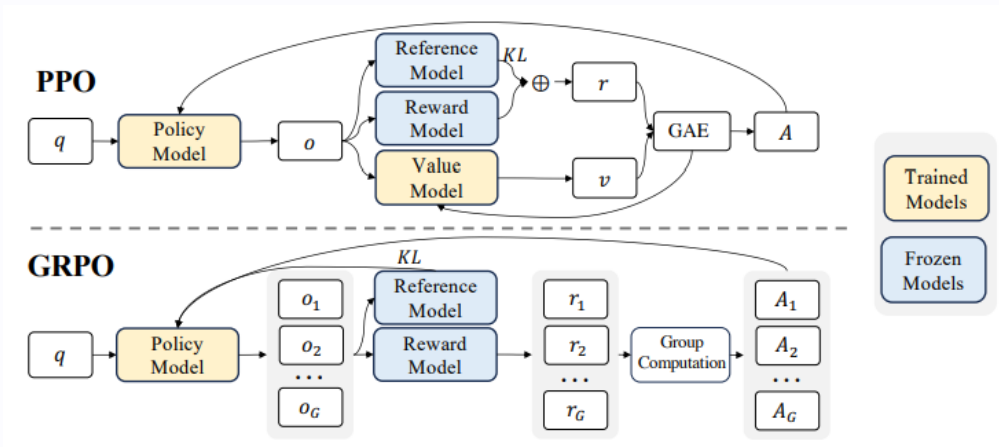
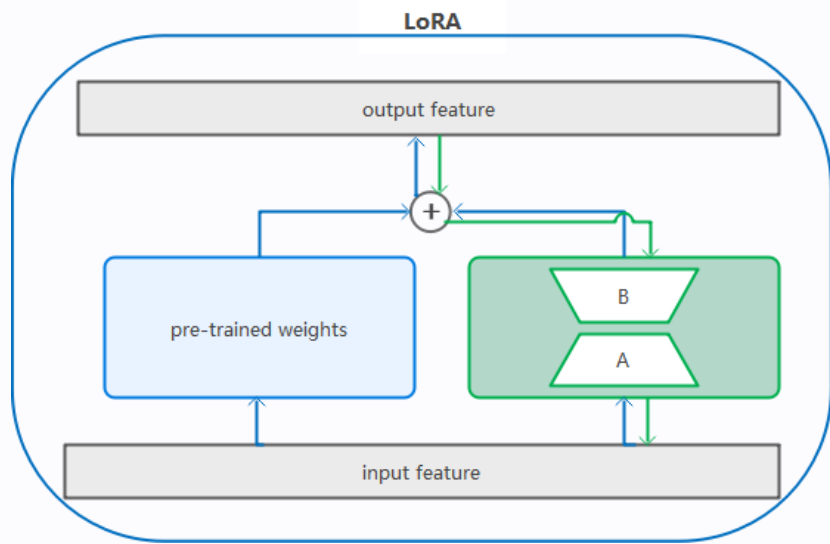
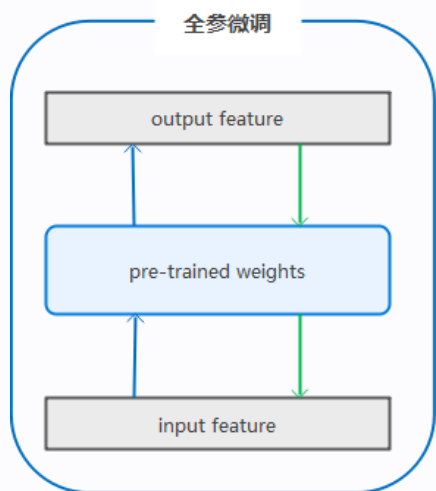
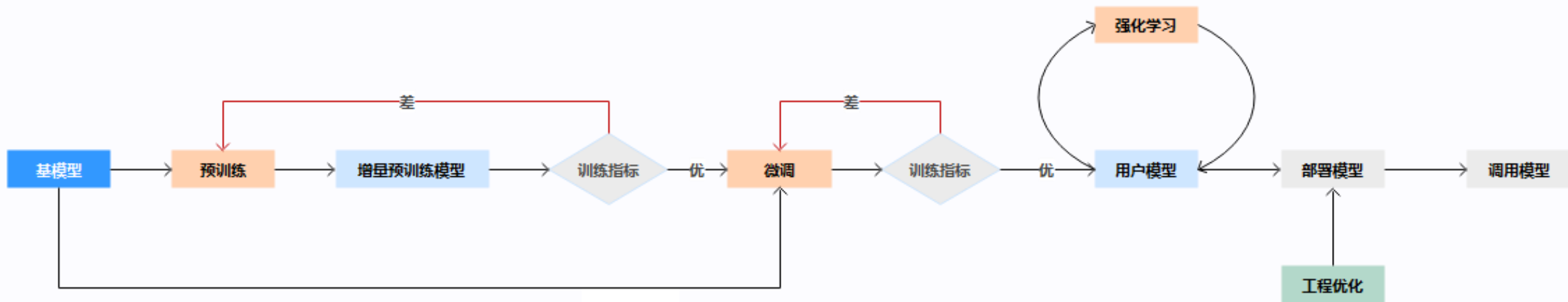
评分规则

3

参考优化手段

4

答疑环节





参考优化手段-数据组织

□ 业界开源数据集+格式组织:

- [OpenR1-Math-220k](#): 220k, 数学, DSR1生成+答案验证
- [LIMO](#): 817条, 数学, 大模型生成+人工优化
- [s1K](#): 1k, 数学+SAT考试, 大模型生成
- [OpenThoughts-114k](#): 114k, 数学、代码、科学、谜题, 大模型生成+答案验证
- [Bespoke-Stratos-17k](#): 17k, 数学、代码、科学、谜题, DSR1生成
- [Sky-T1_data_17k](#): 17k, 数学、代码、科学、谜题, 大模型生成
- ...

□ 组织数据需要注意:

- 质量
- 多样性
- 指令遵从
- 添加通用数据

□ 使用大模型生成数据:

prompts:

You are creating question-answer pairs for fine-tuning a legal assistant.

Focus on technical legal concepts, precedents, and statutory interpretation.

Below is a chunk of text about: {summary}...

Create {num_pairs} high-quality question-answer pairs based ONLY on this text.

Return ONLY valid JSON formatted as:

```
[
  {
    "question": "Detailed legal question?",
    "answer": "Precise legal answer."
  },
  ...
]
```

参考优化手段-解码参数

do_sample

- 值为True时，模型会根据下一个token的概率分布进行采样
- 值为False时，模型会选择概率最大的 token 作为输出。

Temperature (温度)

- 温度主要用于控制模型输出的随机性和创造性。温度越高，输出的随机性和创造性越高；温度越低，输出结果越可以被预测，确定性相对也就越高。
- 可根据真实的任务类型进行调整。一般来说，如果目标任务的需要生成更具创造性的内容，可以使用较高的温度，反之如果目标任务的需要生成更为确定的内容，可以使用较低的温度。



Top k 和 Top p

- top_k 参数控制生成输出时，只考虑概率最高的 top_k 个 token
- top_p 参数与 top_k 参数类似，区别在于 top_p 参数控制生成输出时，只考虑累计概率分布在前 top_p 的 token。



Frequency penalty 和 Presence penalty:

- 频率惩罚 (frequency penalty) 让token每次在文本中出现都受到惩罚。这可以阻止重复使用相同的token/单词/短语，同时也会使模型讨论的主题更加多样化，更频繁地更换主题。
- 存在惩罚 (presence penalty) 是一种固定的惩罚，如果一个 token 已经在文本中出现过，就会受到惩罚。这会导致模型引入更多新的token/单词/短语，从而使其讨论的主题更加多样化，话题变化更加频繁，而不会明显抑制常用词的重复。



参考优化手段-Prompt工程

□ Few-shot 小样本学习

小样本学习会先提供一些关于任务的示例来构造提示。每个示例都包含完整的输入和输出。模型可以通过这些演示更好地理解测试者的意图以及所需要答案的格式。

S 问：中国的首都在哪里？
答：北京
问：意大利的首都在哪里？
答：罗马
问：法国的首都在哪里？
答：

 巴黎。

□ CoT (思维链)

思维链本质就是生成一系列描述性的短句来作为提示、指导模型逐步推理。思维链对于复杂的推理任务+大语言模型的组合有很大的提升效果。

Few-shot Cot:

Question: Tom and Elizabeth have a competition to climb a hill. Elizabeth takes 30 minutes to climb the hill. Tom takes four times as long as Elizabeth does to climb the hill. How many hours does it take Tom to climb up the hill?

Answer: It takes Tom $30 \times 4 = 120$ minutes to climb the hill. It takes Tom $120/60 = 2$ hours to climb the hill. So the answer is 2.

===

Question: Jack is a soccer player. He needs to buy two pairs of socks and a pair of soccer shoes. Each pair of socks cost \$9.50, and the shoes cost \$92. Jack has \$40. How much more money does Jack need?

Answer: The total cost of two pairs of socks is $9.50 \times 2 = 19$. The total cost of the socks and the shoes is $19 + 92 = 111$. Jack need $111 - 40 = 71$ more. So the answer is 71.

===

Question: Marty has 100 centimeters of ribbon that he must cut into 4 equal parts. Each of the cut parts must be divided into 5 equal parts. How long will each final cut be?

Answer:

Zero-shot Cot:

Question: Marty has 100 centimeters of ribbon that he must cut into 4 equal parts. Each of the cut parts must be divided into 5 equal parts. How long will each final cut be?

Answer: Let's think step by step.



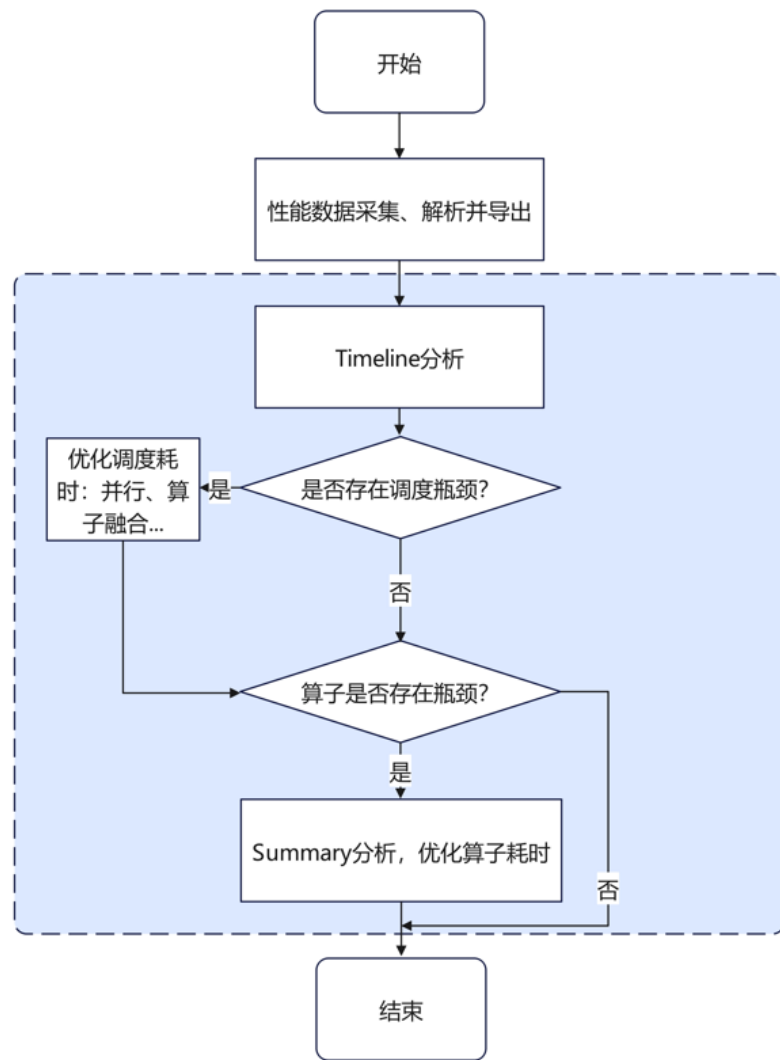
参考优化手段-性能抓取与分析

端到端性能分析流程：

❑ 抓取性能数据：[Ascend PyTorch Profiler接口采集](#)

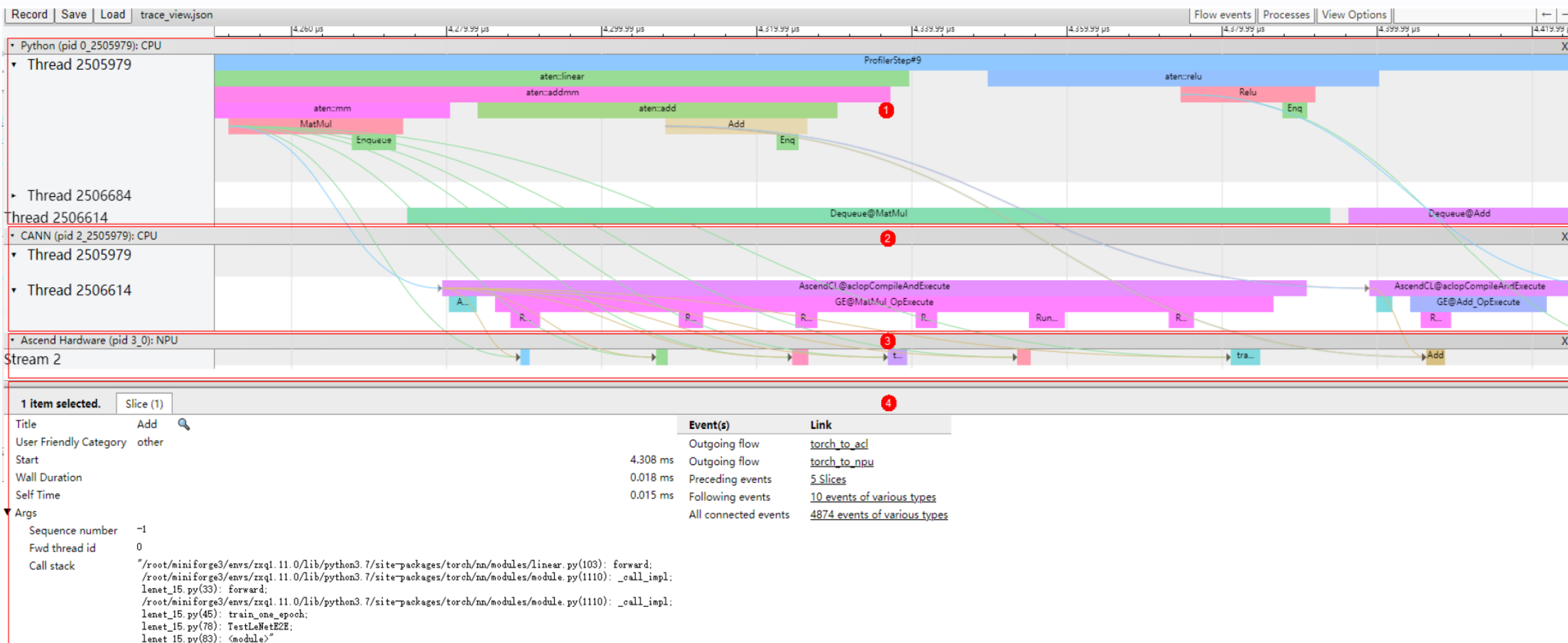
❑ 基础的性能分析：

- timeline：上层API+Host层+Device层算子执行流水；基于TensorBoard可视化或者Chrome插件打开（使用chrome://tracing，然后将文件拖入空白处）；
- summary：可以获取到详细的算子执行耗时信息，主要为*.csv文件。





参考优化手段-性能抓取与分析



①：应用层数据，包含上层应用算子的耗时信息

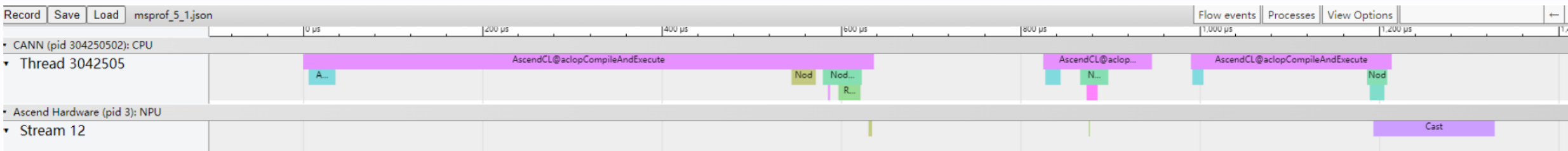
②：CANN数据，包含ACL、Runtime以及Node（算子）耗时信息

③：底层NPU数据，包含Ascend Hardware下各个stream任务流的耗时数据、HCCL和Overlap Analysis通信数据

④：显示图中各算子、接口的详细信息



参考优化手段-性能抓取与分析



□ 步骤1：分析前后处理时间

1. 大模型推理端到端的时间 = 前后处理 + 模型推理时间。前后处理通常是一系列简单小算子的计算，如：输入Mask生成，后处理采样等。通常前后处理小算子在torch/torch_npu上执行，对于小算子数目较多的情况下，会存在调度时间过长的时间。
2. 参考优化手段：将前后处理操作合并到加速库Model层来减少调用时间。

□ 步骤2：分析模型推理流水

1. 确认整网是否存在算子调度瓶颈，常见问题有：Device侧算子空间过大/调度时间过长等（KernelLaunch、数据依赖）；
2. 确认多卡/多芯片之间调度流水同步，常见问题有：多线程间CPU耗时不一致，多芯片间算子/调度时间不一致等；
3. 参考优化手段：绑核，解决多卡的算子调度不齐导致的性能问题。



参考优化手段-性能抓取与分析

步骤3：分析算子执行耗时

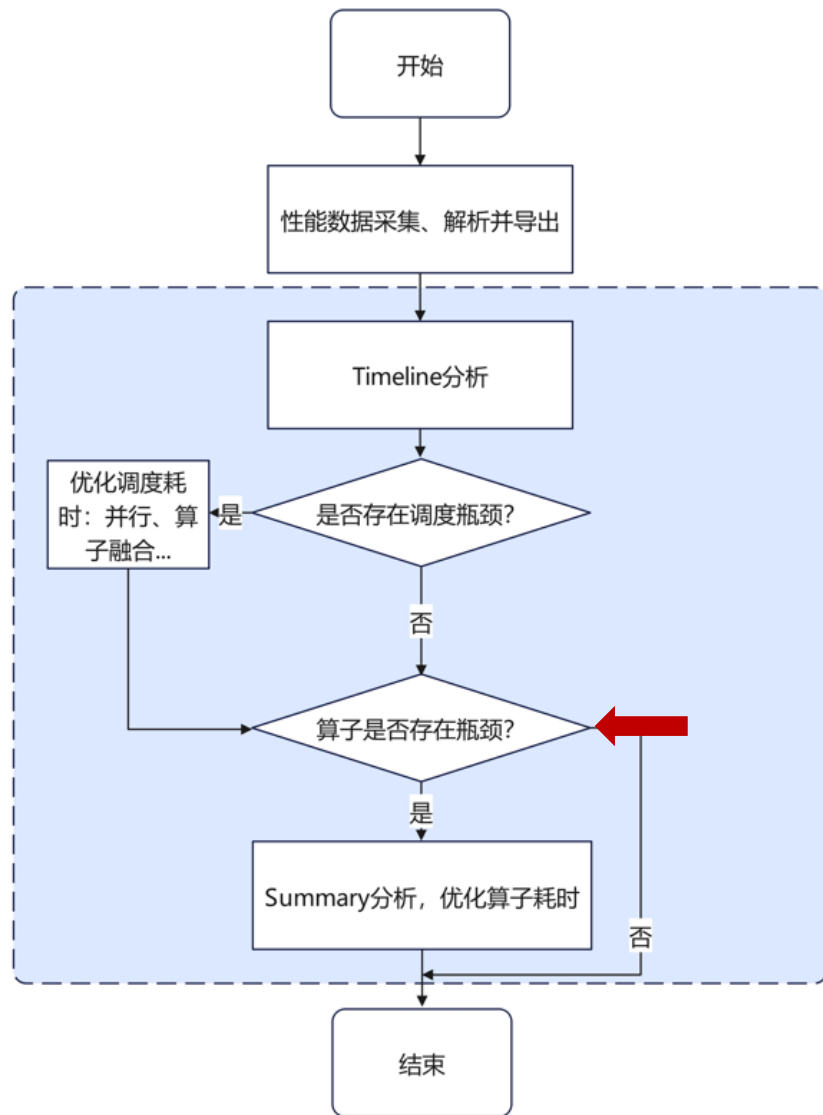
基于op_statistic文件判断瓶颈算子类别（如MatMul算子等）。

OP Type	Core Type	Count	Total Time	Min Time	Avg Time	Max Time	Ratio(%)
MatMulNzF16TB0055Kernel	AI_CORE	28	31968.26	1137.396	1141.723	1146.719	32.4353
MatMulNzF16TB0163Kernel	AI_CORE	84	28084.58	191.5365	334.3403	585	28.4949
AsStridedF16Int640000Kernel	AI_CORE	280	18355.55	1.7708	65.5555	324.4271	18.6237
TransdataNdToNz0006Kernel	AI_CORE	56	5846.797	92.4219	104.4071	117.5	5.9322
TransposeF16Int32F160007Kernel	AI_CORE	28	4972.656	175.8073	177.5949	181.3542	5.0453
TransposeF16Int32F160009Kernel	AI_CORE	28	2976.328	102.7344	106.2974	112.8125	3.0198
BatchMatMulNzF160204Kernel	AI_CORE	28	727.3177	24.8958	25.9756	28.8021	0.7379
BatchMatMulNzF160099Kernel	AI_CORE	28	579.974	20.2604	20.7134	21.9531	0.5884
TransdataNzToNd0017Kernel	AI_CORE	112	546.4323	2.5781	4.8789	6.7188	0.5544

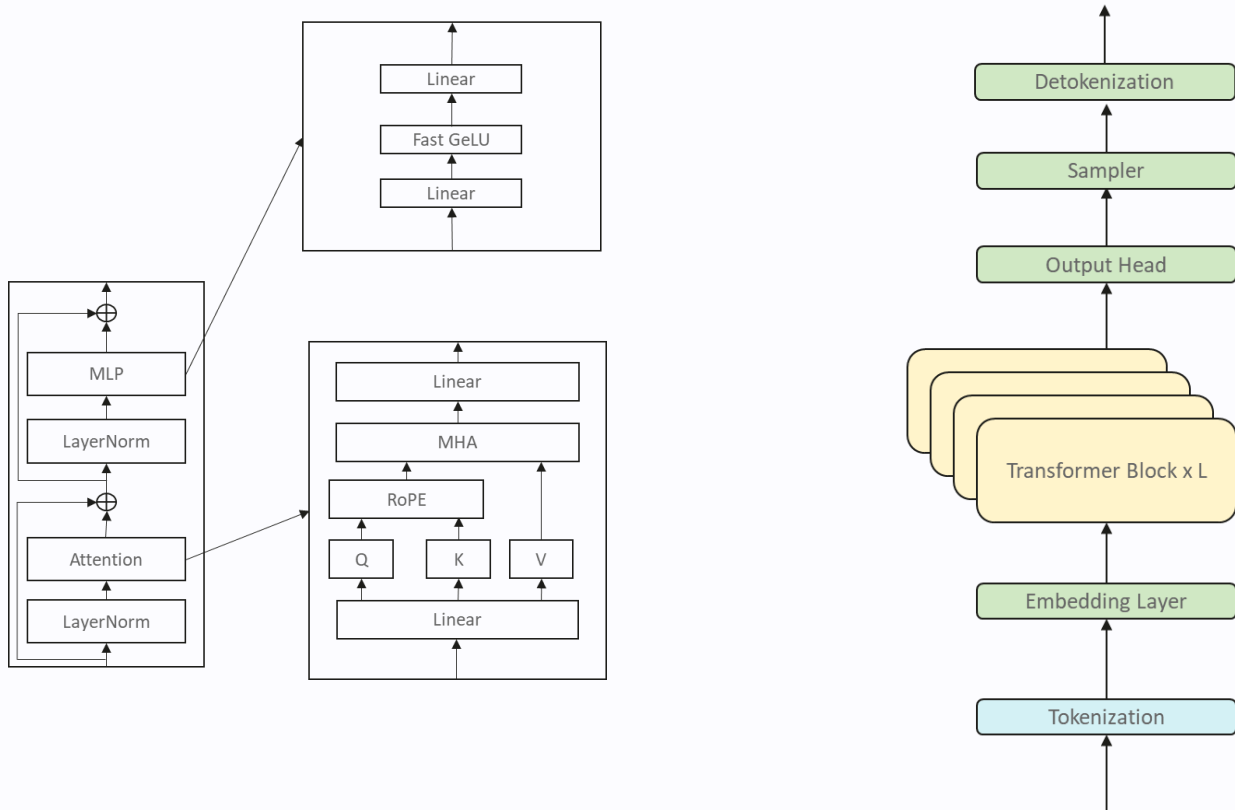
基于op_summary文件判断具体瓶颈算子详细信息。

Op Name	OP Type	Task Ti	Task St	Task Du	Task Wc	mac_tin	mac_rat	scalar	scalar	mtel_ti	mtel_r	mte2_ti	mte2_r	mte3_ti	mte3_r	icache	memory
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1146.719	0.0781	0	0.0593	0	0.0872	0	0.1826	0	0.9404	0	0.0112	0.0005	15.8522
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1146.563	0.0781	0	0.0595	0	0.0874	0	0.1831	0	0.9397	0	0.0091	0.0005	15.802
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1144.983	0.0781	0	0.0596	0	0.0877	0	0.1835	0	0.9387	0	0.009	0.0005	15.7635
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1144.427	0.0781	0	0.0598	0	0.0879	0	0.1839	0	0.9386	0	0.0092	0.0005	15.7272
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1143.433	0.0781	0	0.0598	0	0.0879	0	0.184	0	0.9386	0	0.0076	0.0005	15.7188
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1143.359	0.0781	0	0.0595	0	0.0875	0	0.1831	0	0.9397	0	0.0086	0.0005	15.7998
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1143.333	0.0781	0	0.0598	0	0.088	0	0.1841	0	0.9397	0	0.0088	0.0005	15.7109
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1142.474	0.0781	0	0.0599	0	0.0881	0	0.1844	0	0.9399	0	0.0083	0.0005	15.6904
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1142.422	0.0781	0	0.0598	0	0.0879	0	0.184	0	0.9388	0	0.0073	0.0005	15.7221
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1142.266	0.0781	0	0.0596	0	0.0877	0	0.1836	0	0.9397	0	0.0081	0.0005	15.7604
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1142.24	0.0781	0	0.0596	0	0.0877	0	0.1836	0	0.9385	0	0.0088	0.0005	15.7556
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1142.135	0.0781	0	0.0597	0	0.0877	0	0.1837	0	0.9401	0	0.0082	0.0005	15.7576
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1141.953	0.0781	0	0.0596	0	0.0876	0	0.1835	0	0.94	0	0.009	0.0005	15.7697
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1141.823	0.0781	0	0.0595	0	0.0876	0	0.1833	0	0.94	0	0.0072	0.0005	15.7871
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1141.537	0.0521	0	0.06	0	0.0883	0	0.1848	0	0.9391	0	0.008	0.0005	15.6428
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1141.354	0.0781	0	0.0598	0	0.0879	0	0.1841	0	0.9401	0	0.0087	0.0005	15.7217
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1141.198	0.1042	0	0.0598	0	0.0879	0	0.184	0	0.9397	0	0.0098	0.0005	15.7201
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1141.146	0.0781	0	0.0596	0	0.0877	0	0.1835	0	0.9388	0	0.0085	0.0005	15.7633
MatMulNzF16TB0055Kernel	MatMulNzF16TB0055Kernel	AI_CORE	1E+11	1140.99	0.0781	0	0.0598	0	0.0879	0	0.184	0	0.9387	0	0.0073	0.0005	15.7211

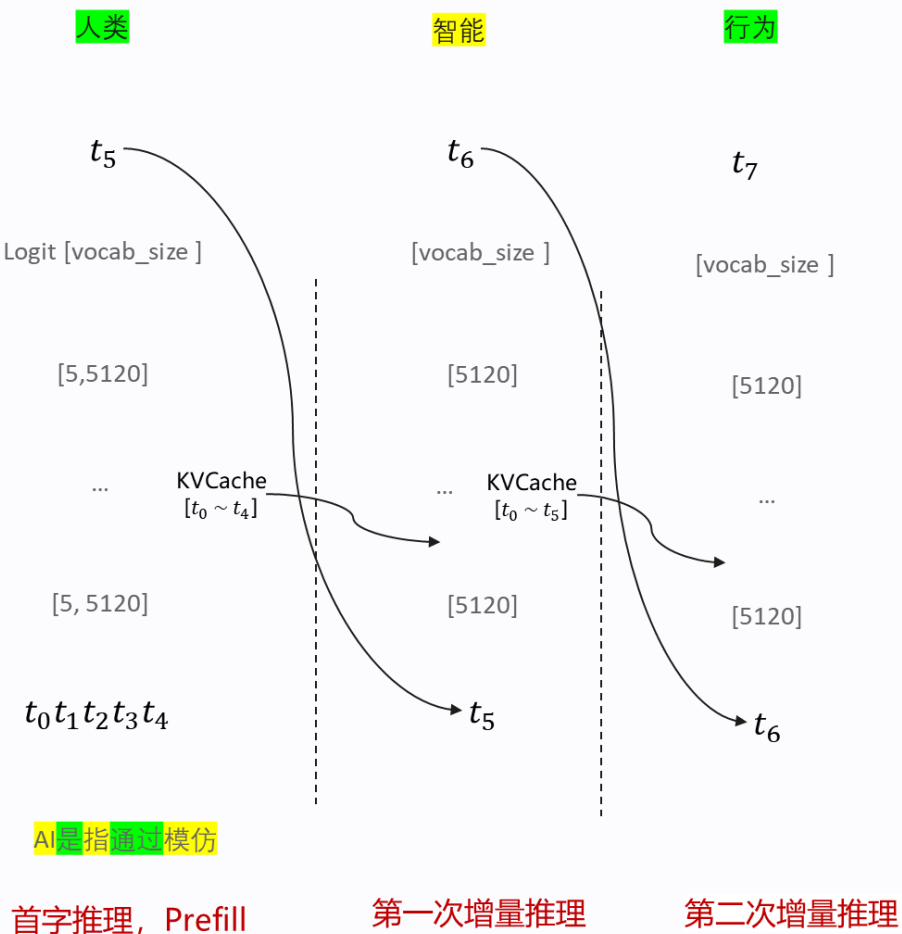
例如MatMul算子在大模型场景下受限于带宽，则mte2_ratio（数据搬运时间）需要大于90%，否则需要优化具体Shape的算子性能。



Transformer Block



AI是指通过模仿人类智能行为来完成的技术。

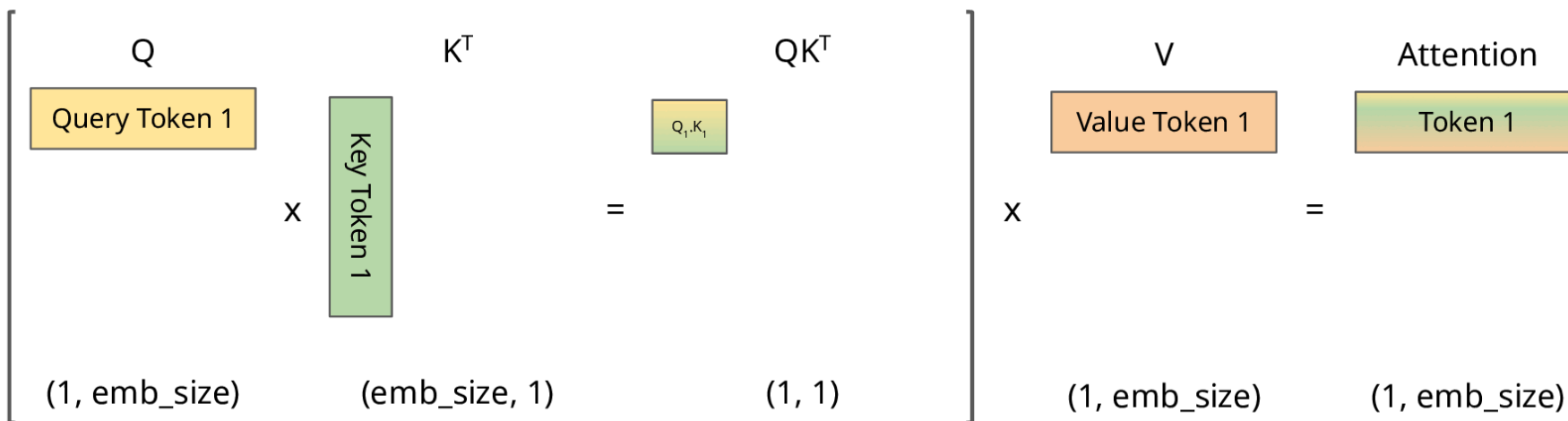


AI是指通过模仿

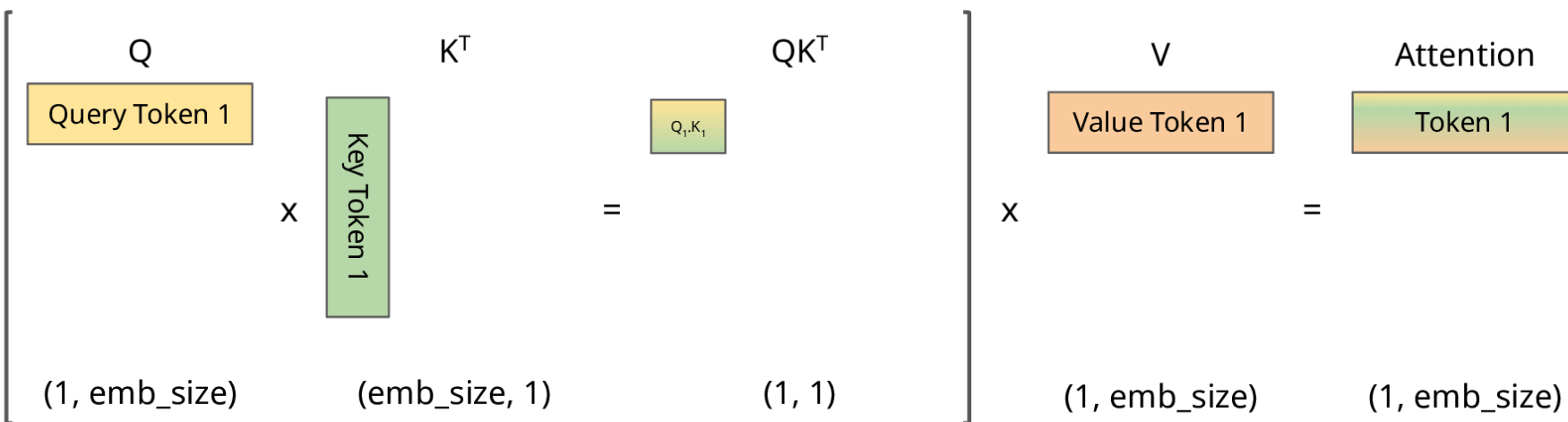
参考优化手段-KVCache

Step 1

Without
cache



With
cache

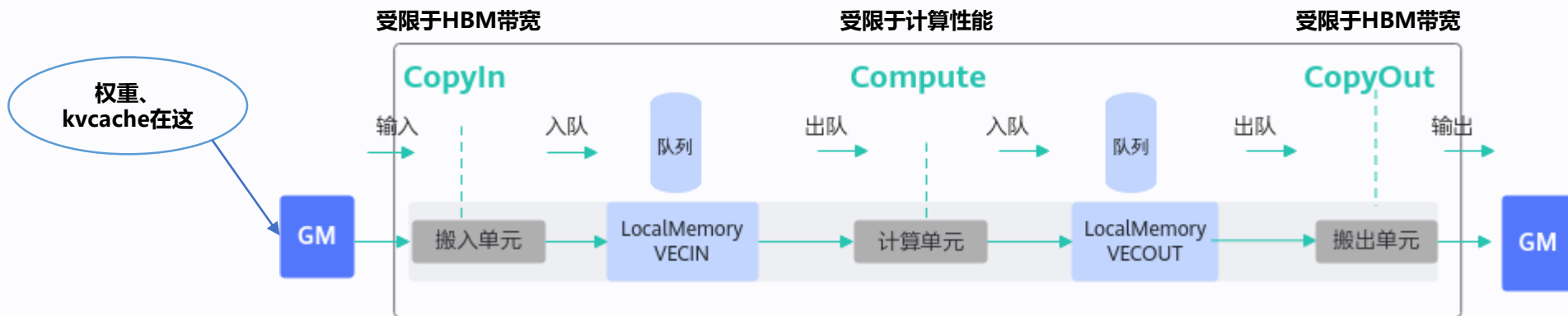
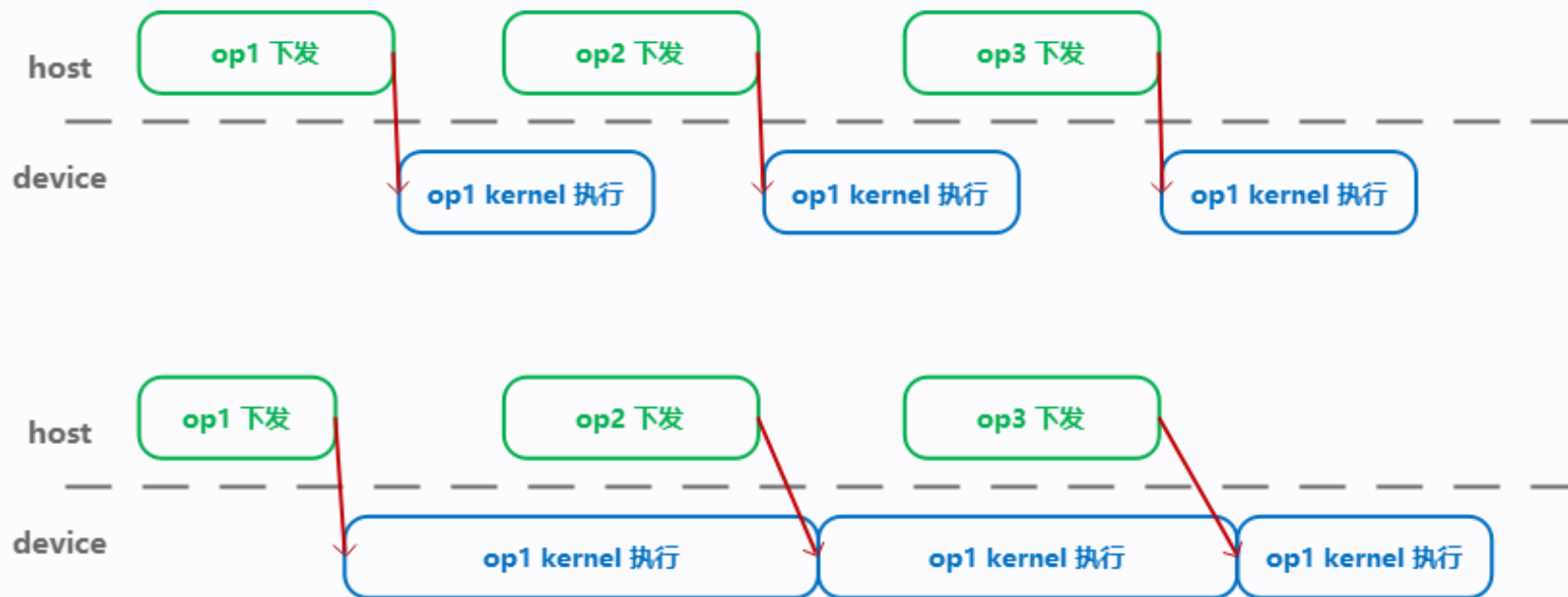


Values that will be masked Values that will be taken from cache

参考优化手段-性能优化

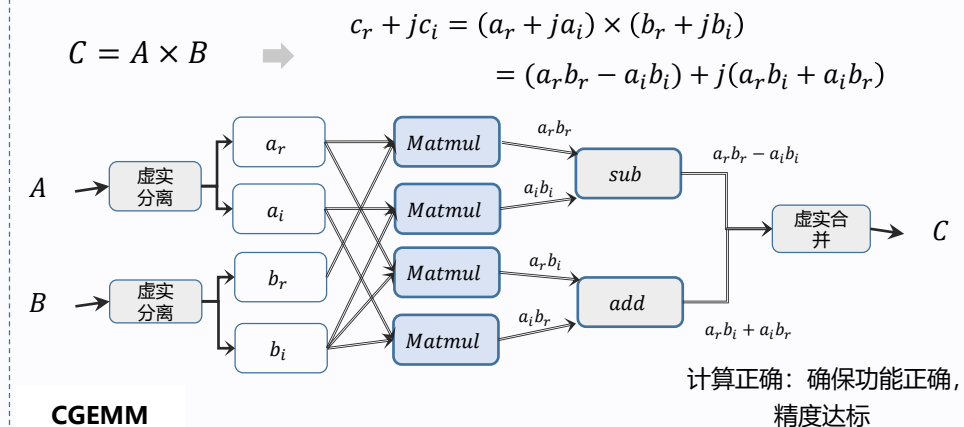


参考优化手段-性能优化-算子优化

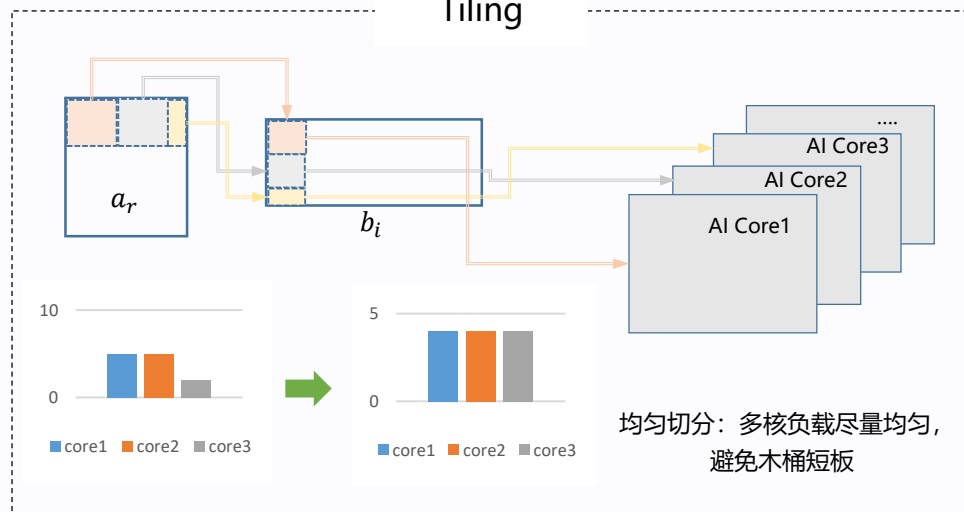


参考优化手段-性能优化-算子优化

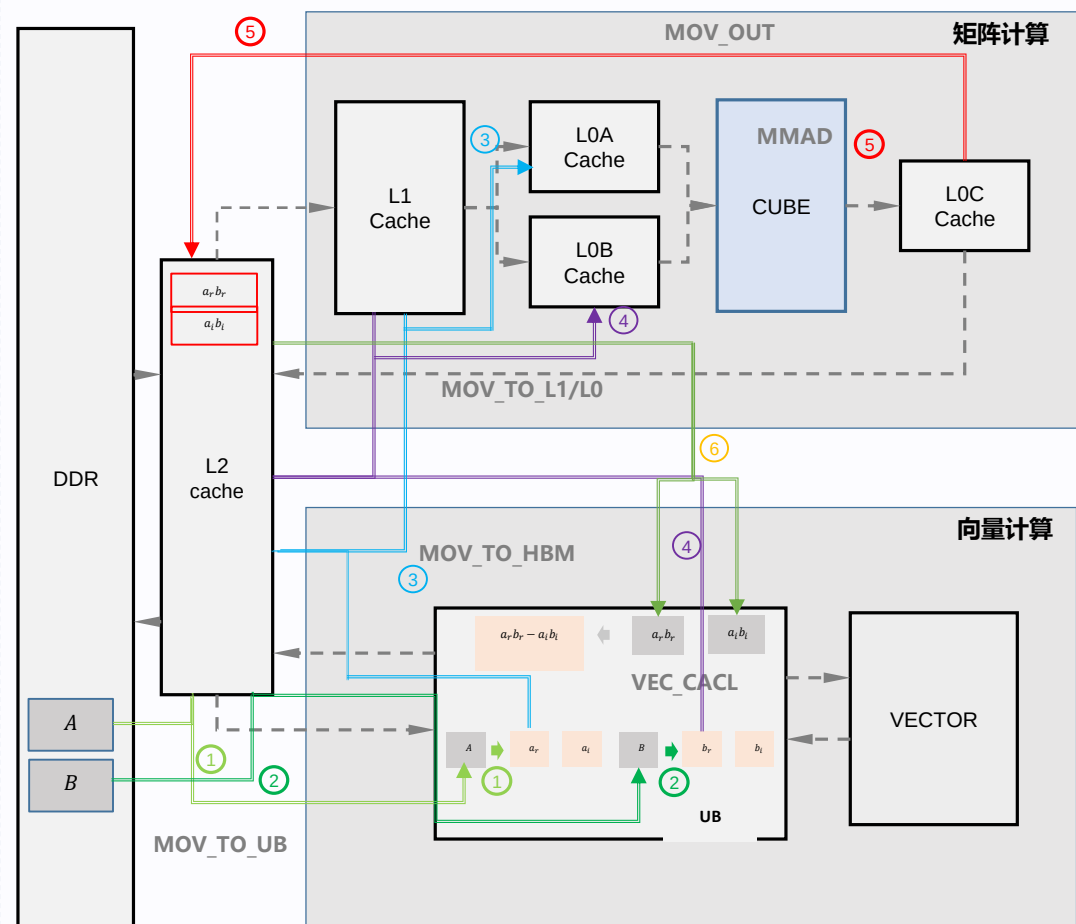
计算方法



Tiling

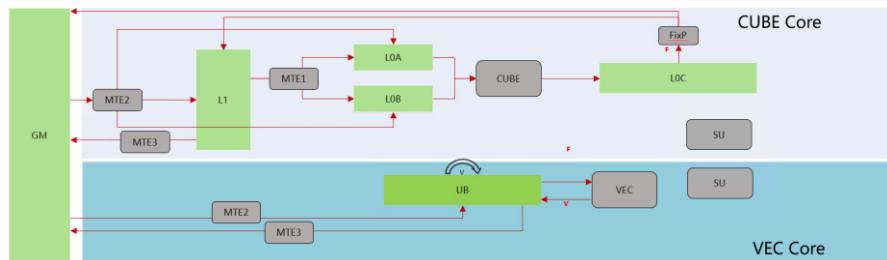


计算流水

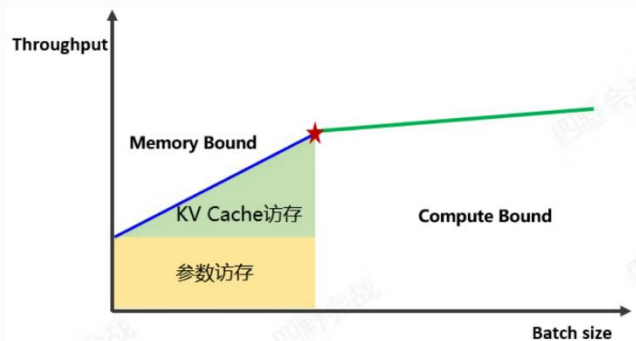




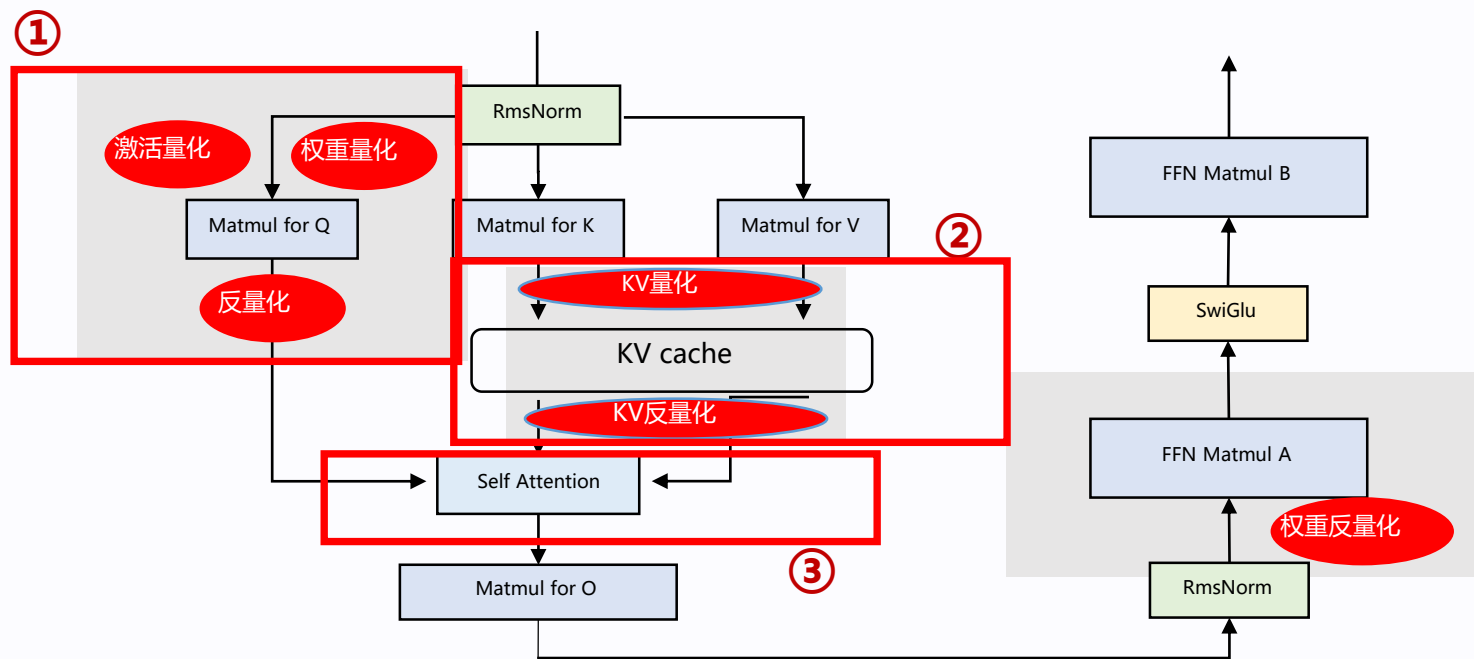
参考优化手段-性能优化-量化

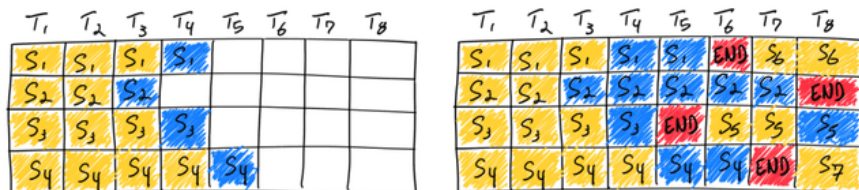


现阶段架构属于CV(Cube和vector)分离架构，执行过程会有更多的访存



昇腾卡在访存上弱于GPU的性能，基于MSD（以定点代浮点）能够减少访存开销





(a) Baseline iteration-level scheduling

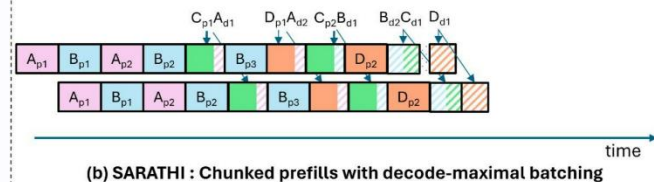
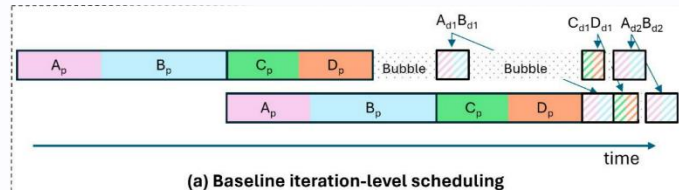
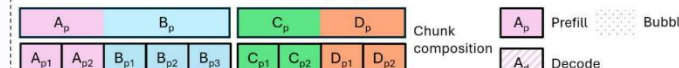


Diagram illustrating the chunk composition for the proposed method. The top row shows a sequence of tokens: A_p (pink), B_p (light blue), C_p (green), and D_p (orange). The bottom row shows the corresponding chunk composition: A_{p1} (pink), A_{p2} (pink), B_{p1} (light blue), B_{p2} (light blue), B_{p3} (light blue), C_{p1} (green), C_{p2} (green), D_{p1} (orange), and D_{p2} (orange). To the right, a legend indicates that A_p is the Prefill chunk, B_p is the Bubble, C_p is the Decode chunk, and D_p is the Decode chunk.



(a) Auto-regressive Decoding

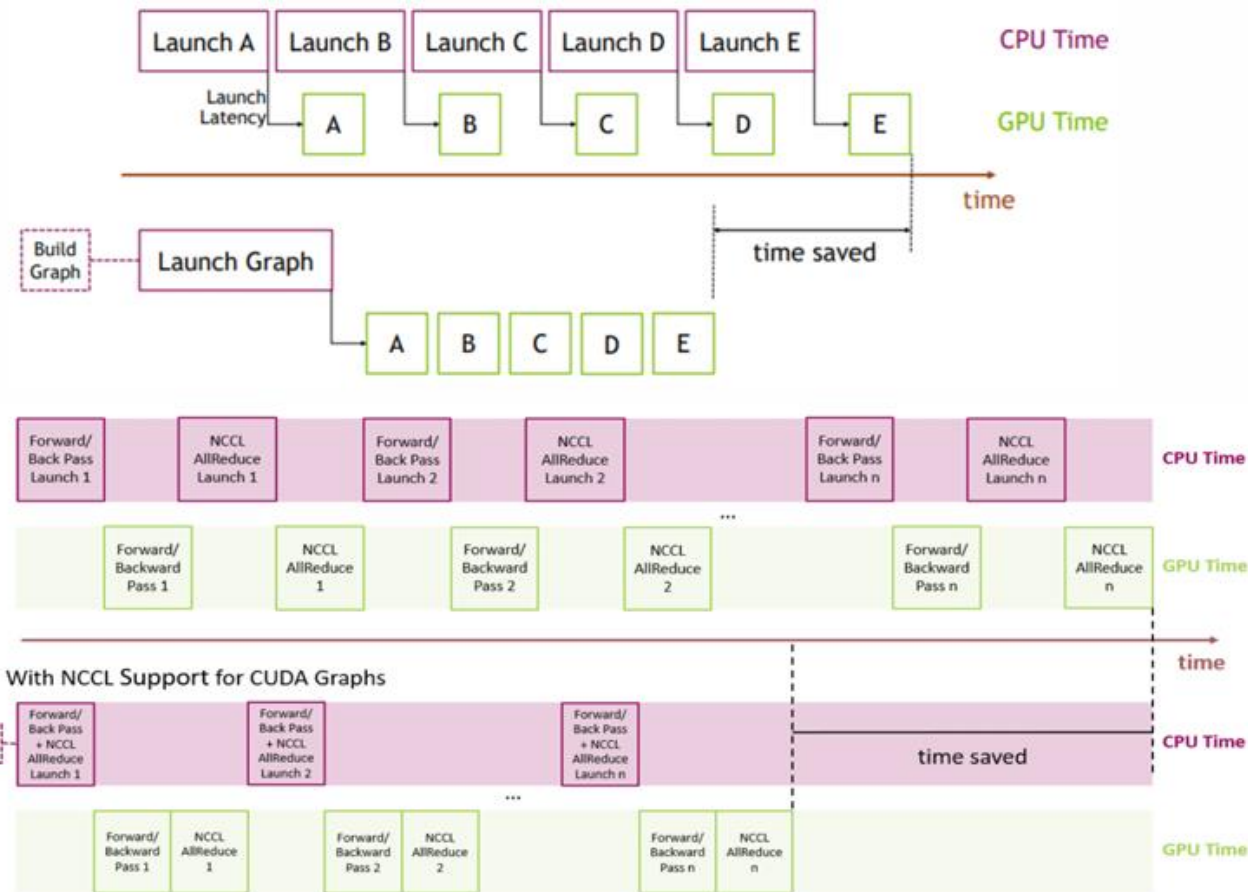
(b) Speculative Decoding

③投机推理

③**投机推理**：首先利用一个更高效的近似模型生成初步的Token序列作为“投机前缀”，然后在目标大模型上并行计算这些Token，最终确保输出的分布与原模型一致。

参考优化手段-性能优化-运行模式

- 单算子模式**：当调用一个操作时，该操作会被立刻执行并返回结果。单算子模式运算路径是动态构建的，操作被逐一提交和执行。这意味着单算子模式运行速度较慢。
- 图模式**：图的结构和运算路径在执行前已经完全构建好，无法动态修改，整个图执行结束才会有返回输出；代码会被编译为中间表示，可以进一步被优化（常量折叠、图融合等），从而提高运行效率。
- CUDA graph**：提供了一种通过单个CPU操作启动多个内核操作的机制，来减少启动开销，从而提高运行效率。



目录



1

赛题解读

2

评分规则

3

参考优化手段

4

答疑环节



答疑环节

选手自行提问

THANK YOU

